

Automated Failure Detection & Alert Notification for Containerized Applications

DevOps IA-2 Case Study · Tool: Prometheus and Alertmanager

Yashasvi Gupta (16010123341)
Shweta Karandikar (16010123329)
Aditi Agrawal (16010123018)

Division faculty: SCP · October 2026

1. Abstract
2. Introduction
 - 2.1 The problem
 - 2.2 Objectives
3. Tool selection
 - 3.1 What we needed
 - 3.2 Candidates
 - 3.3 Why Prometheus and Alertmanager
4. Understanding the tool
 - 4.1 Prometheus
 - 4.2 Alertmanager
5. Practical demonstration
 - 5.1 Setup
 - 5.2 Three angles of monitoring
 - 5.3 Rules
 - 5.4 Alertmanager configuration
 - 5.5 Running the failure scenarios
6. Comparison and evaluation
 - 6.1 Detection and recovery times
 - 6.2 Noise reduction
 - 6.3 False positives
 - 6.4 Resource use
 - 6.5 Comparison with other tools
 - 6.6 Strengths and limitations we observed
7. Testing and continuous integration
8. Conclusion
9. References

1. Abstract

Containers fail in ways that are easy to miss: a process hangs while its container still shows as “running”, memory creeps towards the limit until the kernel kills the process, a database dies and the web service keeps answering with errors. This case study explores **Prometheus** and its companion **Alertmanager** as a way to detect such failures automatically and notify the right people. We built a small containerized order service, OrderFlow, and surrounded it with a monitoring stack of ten containers: white-box application metrics, a black-box prober, container-level metrics from cAdvisor, a Redis exporter, Prometheus with recording and alert rules, and Alertmanager delivering to an email inbox and a chat channel. We injected nine kinds of failure and measured how long each took to reach a person. All nine failures were detected: the first notification arrived between 16 and 90 seconds after the fault (median 60 s), and critical alerts reached the on-call inbox in 26 to 59 seconds. Alertmanager’s grouping and inhibition reduced a database outage that triggered three alerts in Prometheus to a single notification about the root cause, and a 10-second blip produced no page at all. We compare the tool with Nagios, Zabbix, Datadog, Grafana Alerting and AWS CloudWatch, and discuss its strengths and limitations.

2. Introduction

2.1 The problem

Containers make it easy to run many small services, but they also make failure quieter. `docker ps` can show a container as *Up* while the application inside it:

- has hung and no longer answers users;
- returns errors because a dependency (here, a Redis database) is gone;
- leaks memory until the kernel’s out-of-memory killer stops it;
- is starved of CPU by its own limit;
- crashes and is restarted by Docker, hiding the problem behind a restart.

Someone needs to notice these within a minute or two, without staring at dashboards, and without being flooded by duplicate or side-effect alerts.

2.2 Objectives

1. Select a monitoring and alerting tool suited to containerized applications and justify the choice.
2. Instrument a real containerized service and monitor it from three angles: white-box, black-box and container-level.
3. Write alert rules for the failures above, and test the rules automatically.
4. Deliver alerts through two channels (email for on-call, chat for the team) with grouping, inhibition, silences and a heartbeat.
5. Inject each failure, measure detection and recovery times, and evaluate the tool against alternatives.

3. Tool selection

3.1 What we needed

Requirement	Why it matters
Works with short-lived containers	Containers come and go; static host lists don't keep up
Time-series metrics with labels	Rates, percentiles and ratios, not only up/down
Expressive alert conditions	e.g. "more than 10% of requests failing for 30 s"
Notification routing	Different people for different severities; no alert storms
Configuration as code	Rules reviewed and versioned in Git, testable in CI
Free and self-hosted	A student project with no budget, running on a laptop

3.2 Candidates

We looked at Nagios Core, Zabbix, Datadog, Grafana Alerting, AWS CloudWatch and Prometheus with Alertmanager (compared in detail in section 6.5). Datadog and CloudWatch are paid, hosted services; CloudWatch is tied to AWS. Nagios is built around checks against a fixed list of hosts. Zabbix is powerful, but configured mainly through its web interface and backed by an SQL database. Grafana Alerting needs a data source such as Prometheus anyway.

3.3 Why Prometheus and Alertmanager

- **Built for containers.** Prometheus is a graduated project of the Cloud Native Computing Foundation (CNCF). It has service discovery for Docker and Kubernetes and a large ecosystem of exporters.
- **PromQL.** One query language for graphs, recording rules and alerts. It can compute rates, ratios and percentiles directly.
- **Alertmanager** adds grouping, inhibition, silences and routing to many receivers (email, Slack, PagerDuty, webhooks).
- **Everything is a YAML file.** Rules can be unit-tested with `promtool`, and Alertmanager routing can be tested with `amtool`.
- **Open source** (Apache 2.0) and light enough to run on a laptop.

4. Understanding the tool

4.1 Prometheus

Prometheus **pulls** metrics: every few seconds it sends an HTTP request to each *target's* `/metrics` page and stores the result. A target that does not answer gets `up = 0`, so "the scrape failed" is itself a signal. Each sample is a number with a metric name and labels, for example:

```
http_requests_total{route="/api/orders", status="500"} 1234
```

Prometheus keeps these samples in its own **time-series database** (TSDB). Programs that don't speak Prometheus's format are covered by **exporters**, which translate other systems' statistics. We use three: the Redis exporter, the blackbox exporter and cAdvisor.

Metric type	Behaviour	Example in our app
Counter	Only increases; read it through <code>rate()</code>	<code>http_requests_total</code> , <code>orders_created_total</code>
Gauge	Goes up and down	<code>app_dependency_up</code> , <code>container_memory_working_set_bytes</code>
Histogram	Counts observations in buckets	<code>http_request_duration_seconds</code> (used for p95)

Recording rules run a query on a schedule and save the result as a new series. **Alerting rules** are queries with a `for` duration: when the query returns something, the alert becomes *pending*; once it has stayed true for the `for` duration it becomes *firing* and is sent to Alertmanager.

4.2 Alertmanager

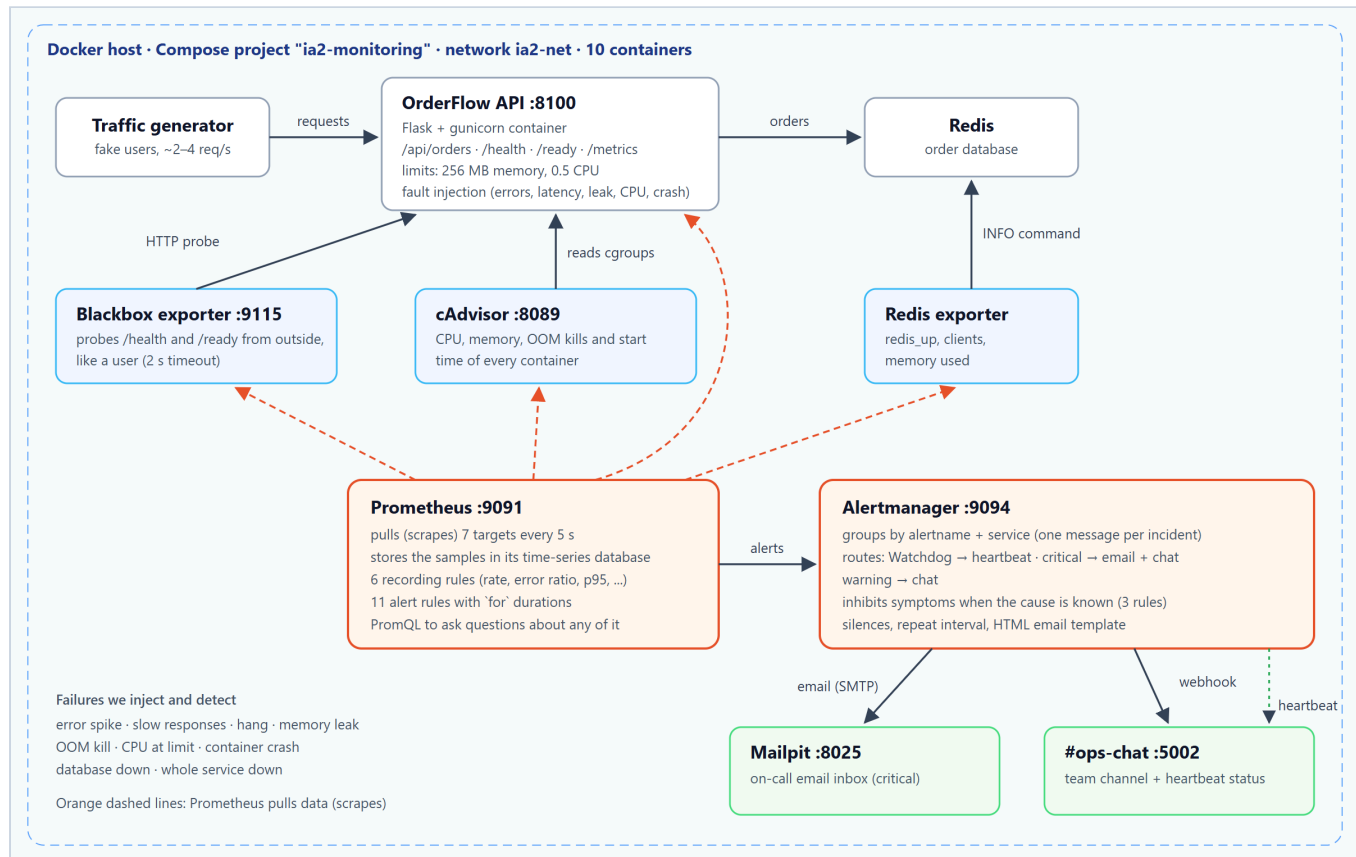
Alertmanager receives firing and resolved alerts and passes them through a fixed pipeline:

1. **Inhibition:** drop alerts that a firing “source” alert explains.
2. **Silences:** drop alerts that someone has muted for a time window.
3. **Grouping:** collect alerts with the same grouping labels into one notification, after waiting `group_wait` for related alerts.
4. **Routing:** walk a tree of matchers to choose receivers (with `continue: true` an alert can go to several).
5. **Notification:** render templates and send; repeat after `repeat_interval` if still firing, and send a “resolved” message when it clears.

It also de-duplicates: several Prometheus servers can send the same alert, and Alertmanager notifies once.

5. Practical demonstration

5.1 Setup



Architecture of the demonstration

Everything runs with one command, `docker compose up -d --build`:

Container	Role
ia2-shop-api	OrderFlow (Flask + gunicorn), limited to 256 MB of memory and 0.5 CPU
ia2-redis	Its database
ia2-traffic	Fake users sending a steady mix of reads and orders
ia2-prometheus	Scrapes 7 targets every 5 s, 6 recording + 11 alert rules
ia2-alertmanager	Routing, grouping, inhibition, templates
ia2-blackbox	Probes /health and /ready from outside
ia2-cadvisor	CPU, memory, OOM kills and start time of every container
ia2-redis-exporter	Redis statistics, including redis_up
ia2-mailpit	Local SMTP server and web inbox for the on-call emails
ia2-ops-chat	Our webhook receiver, shown as a chat channel with a heartbeat badge

OrderFlow

A small containerized order service, watched by Prometheus and Alertmanager

API up

Redis up

Place an order

Item, e.g. Notebook

1

Order

Recent orders

#	Item	Qty	Placed
68	Notebook	2	1:21:03 AM
67	Water bottle	5	1:21:03 AM
66	Backpack	1	1:21:02 AM
65	Notebook	5	1:21:00 AM
64	Pen	1	1:20:59 AM
63	Water bottle	5	1:20:58 AM
62	Notebook	4	1:20:58 AM
61	Water bottle	2	1:20:58 AM

Fault injection

Break the service on purpose and watch Prometheus detect it and Alertmanager notify the on-call email and the ops chat. Alerts arrive 20-60 seconds later, by design.

Slow responses (1.5 s)

Hang (3 s. probe times out)

50% of requests fail

Memory leak (stops near limit)

Run out of memory (OOM kill)

Burn CPU for 2 min

Crash the container

Reset all faults

Database outage: run `docker stop ia2-redis` (and `docker start ia2-redis` to recover).

latency 0 ms · error rate 0% · leaked 0 MB · CPU burn off

[Prometheus](#) [Alertmanager](#) [Ops chat](#) [On-call email \(Mailpit\)](#) [cAdvisor](#) [/metrics](#)

OrderFlow with its fault-injection panel

Prometheus

Query Alerts Status > Target health

Select scrape pool Filter by target health Filter by endpoint or labels

alertmanager

1 / 1 up

Endpoint	Labels	Last scrape	State
http://alertmanager:9093/metrics	instance="alertmanager:9093" job="alertmanager"	1.949s ago 9ms	UP

blackbox-http

2 / 2 up

Endpoint	Labels	Last scrape	State
http://blackbox:9115/probe module="http_2xx" target="http://shop-api:8000/health"	app="orderflow" endpoint="health" instance="http://shop-api:8000/health" job="blackbox-http" service="shop-api"	887ms ago 3ms	UP
http://blackbox:9115/probe module="http_2xx" target="http://shop-api:8000/ready"	app="orderflow" endpoint="ready" instance="http://shop-api:8000/ready" job="blackbox-http" service="shop-api"	1.921s ago 8ms	UP

cadvisor

1 / 1 up

Endpoint	Labels	Last scrape	State
http://cadvisor:8080/metrics	instance="cadvisor:8080" job="cadvisor"	6.389s ago 16ms	UP

prometheus

1 / 1 up

Endpoint	Labels	Last scrape	State
http://localhost:9090/metrics	instance="localhost:9090" job="prometheus"	4.776s ago 13ms	UP

Prometheus scraping all seven targets

7 / 22

5.2 Three angles of monitoring

White-box. OrderFlow uses the `prometheus_client` library to expose request counts by route and status, a latency histogram, orders created, and whether Redis answers. Fault injection deliberately leaves `/metrics` unaffected. That matters in the “hang” scenario: Prometheus can still scrape the process, so `up` stays 1, while users time out.

Black-box. The blackbox exporter requests `/health` and `/ready` with a 2-second timeout and reports `probe_success`. Prometheus passes each URL to the exporter with a standard relabelling pattern:

```
- job_name: blackbox-http
  metrics_path: /probe
  params: { module: [http_2xx] }
  static_configs:
    - targets: ["http://shop-api:8000/health"]
  relabel_configs:
    - { source_labels: [__address__], target_label: __param_target }
    - { source_labels: [__param_target], target_label: instance }
    - { target_label: __address__, replacement: blackbox:9115 }
```

Container-level. cAdvisor reads each container’s cgroup statistics, so it sees memory against the container’s limit, CPU against its quota, OOM kills and restarts, even for containers that expose no metrics of their own. cAdvisor reports a large number of labels, so a `metric_relabel_configs` rule keeps only this project’s containers (named `ia2-*`).



cAdvisor

Docker Containers

Docker Containers

Subcontainers

ia2-blackbox (/docker/c51b3eb8b3fd4c875f50a7edad63d083867027a26b03e5e2fb3d51458834a6e6)
ia2-shop-api (/docker/e086833b6c2f84074de2e1fd9d949cf3a258b109380f045f823938251bb40c62)
sensor-dashboard (/docker/f2b03272ca69067d5aa79b0deca15c62b8cef5cd6683a2aad3ae1df5bbafe4e8)
ia2-alertmanager (/docker/5594d6658b58de15e0693570b4b74b4815e8b645eb498ea196e078e69108cc21)
jenkins (/docker/1340b9bcbe7c0820144189a7d1d9dd18d6f6d73f7dad4925733e8f08e91b82ae)
prometheus (/docker/efd947c6087fc731515eaa3375b9a3953dbc2743a5a34764a27634f2985e92)
ia2-redis-exporter (/docker/7da0ba9f90f9832a1cba8ca6b37b03542c86b598e00fef5973d5302765383d3)
alertmanager (/docker/3a6f574d5ac62b4741333ff420fb419fb6daeeaa25e85739a8e1b576f7c1d07db)
alert-receiver (/docker/a5759d179845d39548c2072aca227ff47109041d190de21f34766e1479a89cc9)
ia2-ops-chat (/docker/607ef0f1e73e4f033e93176d30443830aa06bdc2a76284901fd1fbb2a640db80)
ia2-mailnit (/docker/7e99b3e6cdc99d155858beb8f1bef5607639d25ff3a2bc0be414b3a8617fedb0)

cAdvisor finds every container on the machine, including other projects’, hence the `ia2-*` filter

5.3 Rules

Recording rules give short names to the queries used everywhere else:

```
- record: service:http_errors:ratio_rate1m
  expr: |
    ( sum by (app, service) (rate(http_requests_total{job="shop-api", status=~"5.."}[1m]))
      or
      sum by (app, service) (rate(http_requests_total{job="shop-api"}[1m])) * 0 )
    / sum by (app, service) (rate(http_requests_total{job="shop-api"}[1m]))
```

The `or ... * 0` part is something we learned the hard way. Before the first error there is no 5xx series at all, so the ratio showed “no data” instead of 0%.

Alert	Condition	Severity
ServiceDown	<code>up == 0</code> for 30 s	critical
EndpointDown	black-box probe of <code>/health</code> fails for 40 s	critical
DependencyDown	<code>redis_up == 0</code> for 15 s	critical
HighErrorRate	error ratio > 10% for 30 s	critical
ContainerOOMKilled	<code>increase(container_oom_events_total[5m]) > 0</code>	critical
ServiceNotReady	<code>/ready</code> probe fails for 45 s	warning
HighLatencyP95	p95 > 500 ms for 1 min	warning
ContainerMemoryNearLimit	memory > 80% of the limit for 15 s	warning
ContainerCPUThrottled	throttled in > 50% of CPU periods for 30 s	warning
ContainerRestarted	container start time changed in the last 5 min	warning
Watchdog	<code>vector(1)</code> , always firing	none

Each alert carries a `summary`, a `description` and a `runbook` hint (what to check first), which the email template prints.

The rules have unit tests (`promtool test rules`). Each test feeds made-up series and asserts which alerts fire, with their exact text. One test proves that a 10-second outage does **not** fire `ServiceDown`. Another proves that a container without a memory limit is not reported as “near its limit”; without the `> 0` filter, dividing by a zero limit would give infinity.

5.4 Alertmanager configuration

```
route:
  receiver: ops-chat
  group_by: [alertname, service]
  group_wait: 10s
  group_interval: 30s
  repeat_interval: 3h
  routes:
    - matchers: ['alertname="Watchdog"']      # heartbeat, every ~minute
      receiver: heartbeat
    - matchers: ['severity="critical"']        # email on-call, then continue
      receiver: oncall-email
      group_wait: 5s
      continue: true
    - matchers: ['severity=~"critical|warning"']
      receiver: ops-chat
```

Three inhibition rules hide symptoms when their cause is firing: `DependencyDown` hides `HighErrorRate`, `HighLatencyP95` and `ServiceNotReady`; `ServiceDown` hides `EndpointDown` and the rest; `EndpointDown` hides the latency and readiness alerts it causes. For inhibition to work, the cause must already be firing when the symptom arrives. So causes have *shorter* for durations than their symptoms (15 s for `DependencyDown` against 30 s for `HighErrorRate`). We got this ordering wrong at first.

The email receiver uses a custom template, which gives a subject such as `[FIRING x1] HighErrorRate on shop-api` and an HTML body with the summary, details, a “what to do” line and the labels. Routing was checked with `amtool config routes test`: a critical alert reaches `oncall-email` and `ops-chat`, a warning reaches `ops-chat`, and the Watchdog reaches `heartbeat`.

5.5 Running the failure scenarios

Each failure is injected from the app’s fault-injection panel, with `scripts/chaos.ps1`, or with `docker stop` for outages. The figures below come from real runs; measured times for every scenario are in section 6.1.

Failure	How it is injected	What detects it
Error spike	50% of API requests return HTTP 500	HighErrorRate from white-box metrics
Slow responses	+ 1.5 s per request	HighLatencyP95 from the latency histogram
Hang	+ 3 s per request; the probe's 2 s timeout expires	EndpointDown from the black-box probe, while up stays 1
Memory leak	5 MB/s, capped at about 85% of the limit	ContainerMemoryNearLimit from cAdvisor
Out of memory	40 MB/s with no cap	ContainerOOMKilled from cAdvisor's OOM counter
CPU exhaustion	two busy threads for 2 minutes	ContainerCPUTHrottled from cAdvisor
Crash	the process exits and Docker restarts it	ContainerRestarted from cAdvisor
Database outage	<code>docker stop ia2-redis</code>	DependencyDown from the Redis exporter
Full outage	<code>docker stop ia2-shop-api</code>	ServiceDown from up

Mailpit

50
1-19 of 19

Inbox 19

Mark all read

Delete all

alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] DependencyDown on redis Something needs attention: DependencyDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary Redis (o...	2.5 kB	a minute ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] DependencyDown on redis Resolved: DependencyDown OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary Redis (order database)...	2.5 kB	4 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] DependencyDown on redis Something needs attention: DependencyDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary Redis (o...	2.5 kB	4 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] ServiceDown on shop-api Resolved: ServiceDown OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary OrderFlow API is down Deta...	2.5 kB	11 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] ServiceDown on shop-api Something needs attention: ServiceDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary OrderFlow A...	2.5 kB	11 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] ContainerOOMKilled Resolved: ContainerOOMKilled OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary ia2-shop-api ran out...	2.5 kB	14 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] ContainerOOMKilled Something needs attention: ContainerOOMKilled OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary ia2-s...	2.5 kB	19 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] HighErrorRate on shop-api Resolved: HighErrorRate OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary 12.17% of OrderFlow requ...	2.4 kB	20 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] DependencyDown on redis Resolved: DependencyDown OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary Redis (order database)...	2.5 kB	20 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] DependencyDown on redis Something needs attention: DependencyDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary Redis (o...	2.5 kB	20 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] DependencyDown on redis Resolved: DependencyDown OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary Redis (order database)...	2.5 kB	23 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] HighErrorRate on shop-api Something needs attention: HighErrorRate OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary 64.29% of O...	2.5 kB	23 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] DependencyDown on redis Something needs attention: DependencyDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary Redis (o...	2.5 kB	23 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[RESOLVED] EndpointDown on shop-api Resolved: EndpointDown OrderFlow monitoring - 0 firing, 1 resolved Status RESOLVED Summary OrderFlow health check fa...	2.6 kB	38 minutes ago
alertmanager@orderflow.local To: oncall@orderflow.local	[FIRING x1] EndpointDown on shop-api Something needs attention: EndpointDown OrderFlow monitoring - 1 firing, 0 resolved Status FIRING Summary OrderFlow ...	2.6 kB	38 minutes ago

About

The on-call inbox: one email per incident, firing and resolved

Mark unread

Delete

Download

Return to inbox

alertmanager@orderflow.local

To: oncall@orderflow.local

[FIRING x1] DependencyDown on redis

4 minutes ago

[RESOLVED] DependencyDown on redis

4 minutes ago

[FIRING x1] DependencyDown on redis

11 minutes ago

[RESOLVED] ServiceDown on shop-api

11 minutes ago

[FIRING x1] ServiceDown on shop-api

14 minutes ago

[RESOLVED] ContainerOOMKilled

19 minutes ago

[FIRING x1] ContainerOOMKilled

20 minutes ago

[RESOLVED] HighErrorRate on shop-api

20 minutes ago

[RESOLVED] DependencyDown on redis

21 minutes ago

[FIRING x1] DependencyDown on redis

From

<alertmanager@orderflow.local>

To

<oncall@orderflow.local>

Subject

[FIRING x1] DependencyDown on redis

Date

Sat, 3 Oct 2026, 1:22 am (2.5 kB)

Tags

Add tags...

HTML

HTML Source

Text

Headers

Raw

HTML Check 91%

Link Check

Something needs attention: DependencyDown

OrderFlow monitoring · 1 firing, 0 resolved

Status

FIRING

Summary

Redis (order database) is down

Details

The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

What to do

Check the ia2-redis container: `docker ps -a`, `docker start ia2-redis`.

Started

2026-10-02 19:52:12 UTC

Labels

alertname=DependencyDown app=orderflow environment=ia2-demo instance=redis-exporter:9121 job=redis service=redis severity=critical

Sent by Alertmanager. Silence or browse alerts at <http://localhost:9094>

An alert email built from our template, with a “what to do” line

OrderFlow team

ops-chat

Warnings and critical alerts land here. Critical alerts are also emailed to the on-call engineer.

ops-chat

Alerting pipeline healthy: last heartbeat 69 s ago

20 firing · 18 resolved notifications in this channel · page refreshes every 5 s

AM

Alertmanager 19:52:26 UTC · #ops-chat

FIRING DependencyDown · critical · redis

Redis (order database) is down

The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

AM

Alertmanager 19:50:36 UTC · #ops-chat

FIRING ContainerRestarted · warning · ia2-redis

Container ia2-redis restarted

ia2-redis started again in the last 5 minutes (crash, OOM kill or manual restart).

RESOLVED ContainerRestarted · warning · ia2-shop-api

Container ia2-shop-api restarted

ia2-shop-api started again in the last 5 minutes (crash, OOM kill or manual restart).

AM

Alertmanager 19:50:06 UTC · #ops-chat

RESOLVED DependencyDown · critical · redis

Redis (order database) is down

The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

AM

Alertmanager 19:50:06 UTC · #ops-chat

FIRING ContainerRestarted · warning · ia2-redis

Container ia2-redis restarted

ia2-redis started again in the last 5 minutes (crash, OOM kill or manual restart).

AM

Alertmanager 19:49:36 UTC · #ops-chat

FIRING DependencyDown · critical · redis

Redis (order database) is down

The Redis exporter cannot reach Redis. OrderFlow cannot store or read orders.

AM

Alertmanager 19:48:44 UTC · #ops-chat

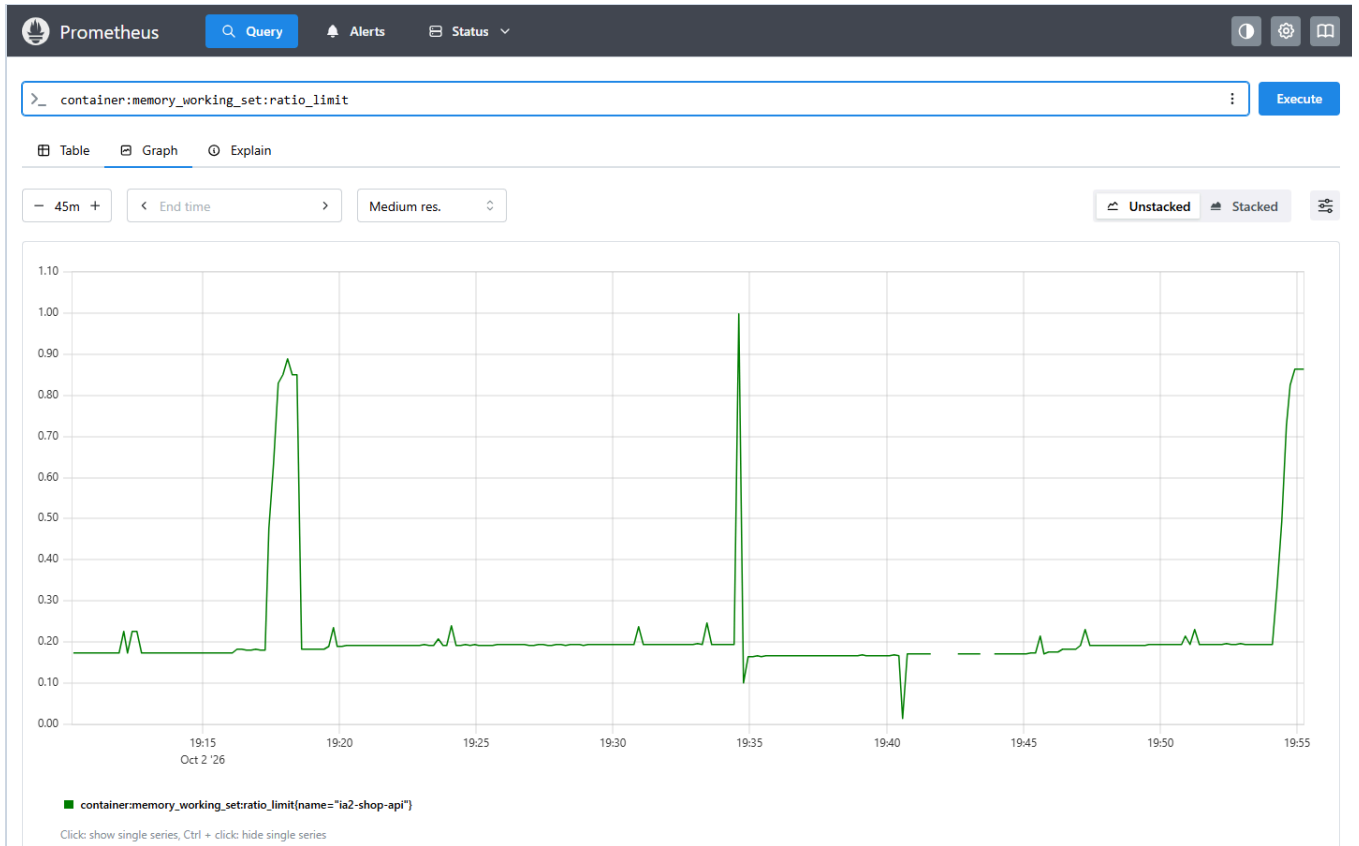
RESOLVED ContainerRestarted · warning · ia2-shop-api

Container ia2-shop-api restarted

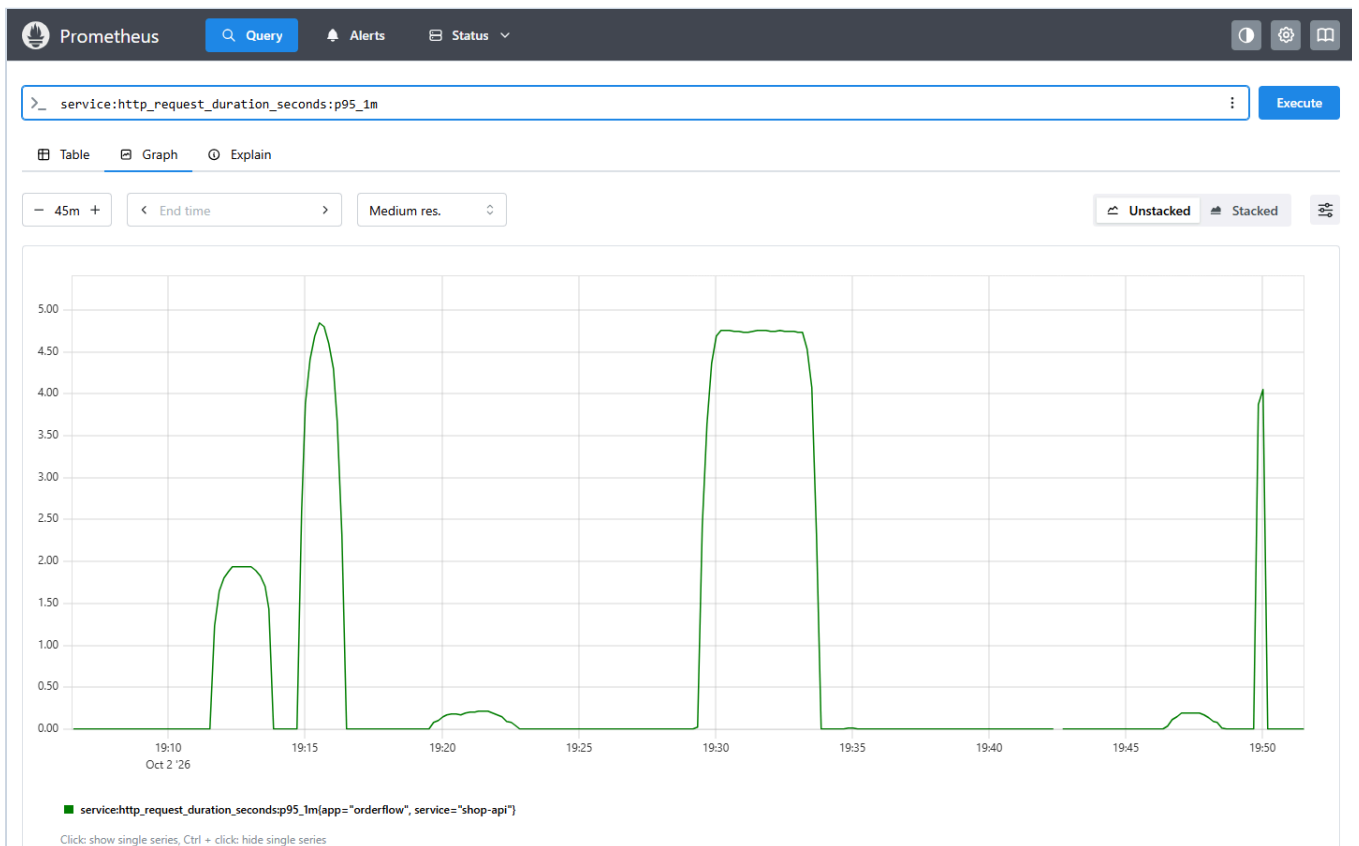
ia2-shop-api started again in the last 5 minutes (crash, OOM kill or manual restart).

The ops chat: alert messages and the heartbeat badge (top right)

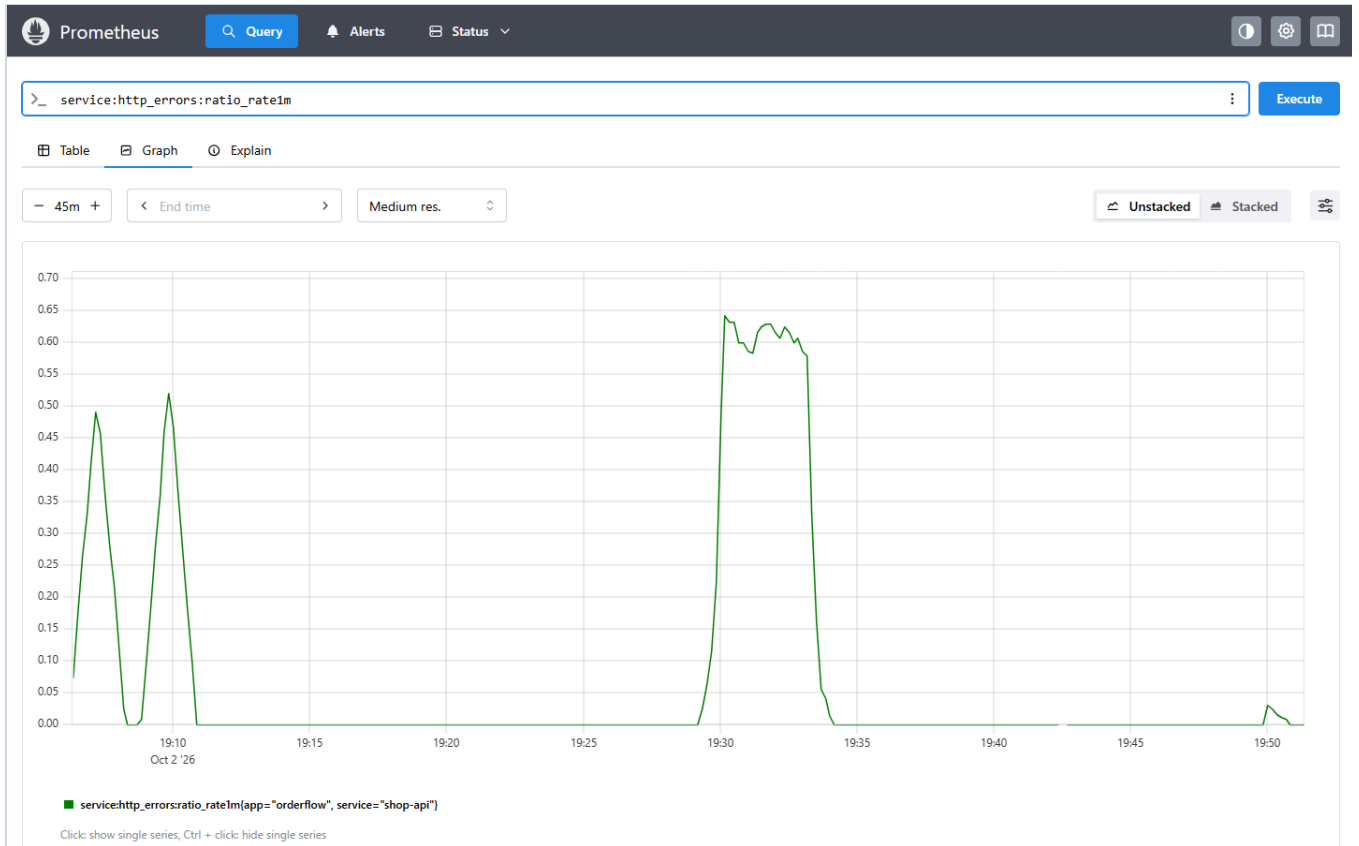
12 / 22



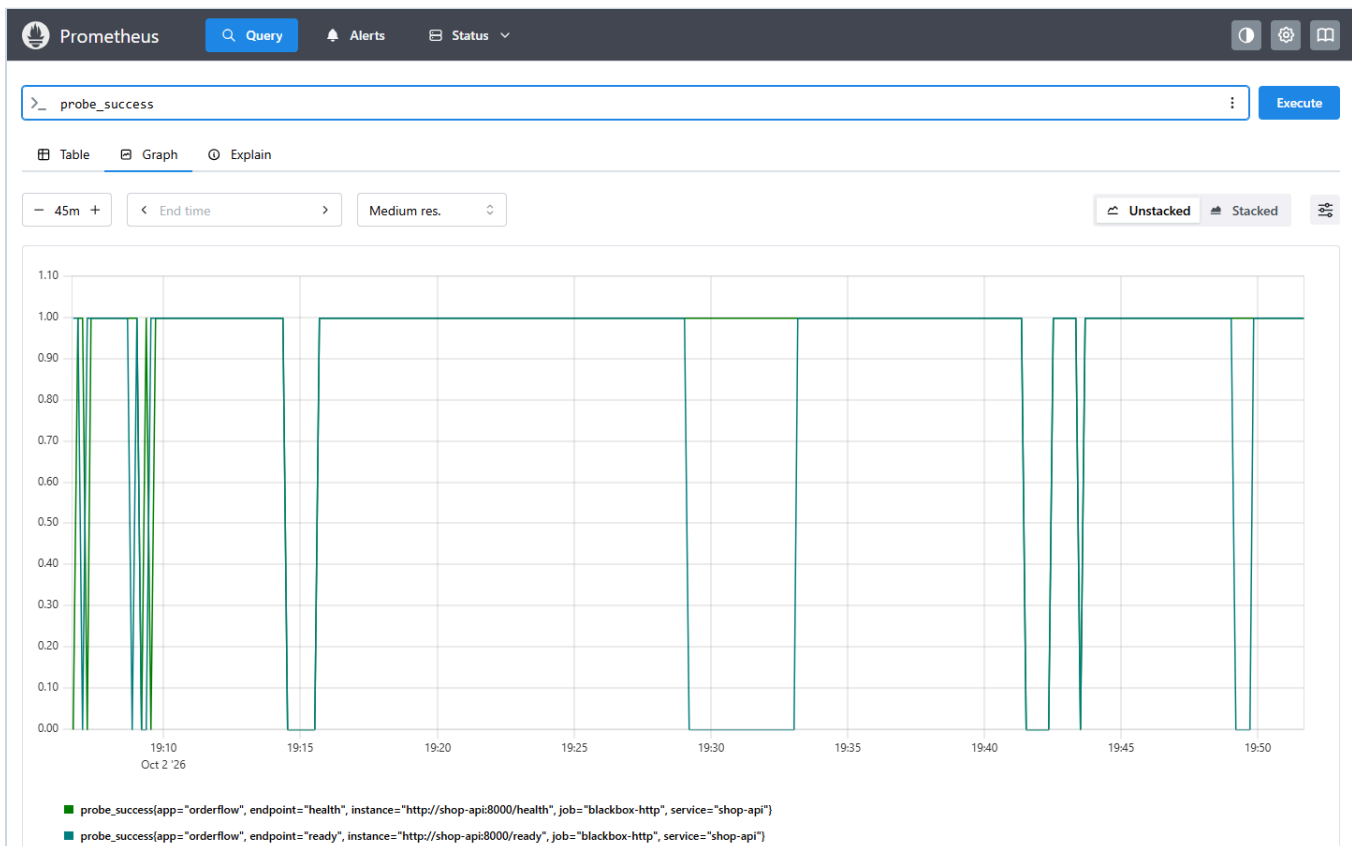
Memory of the OrderFlow container as a share of its limit: the leak plateaus near 85%, and the spike to 100% is the OOM kill



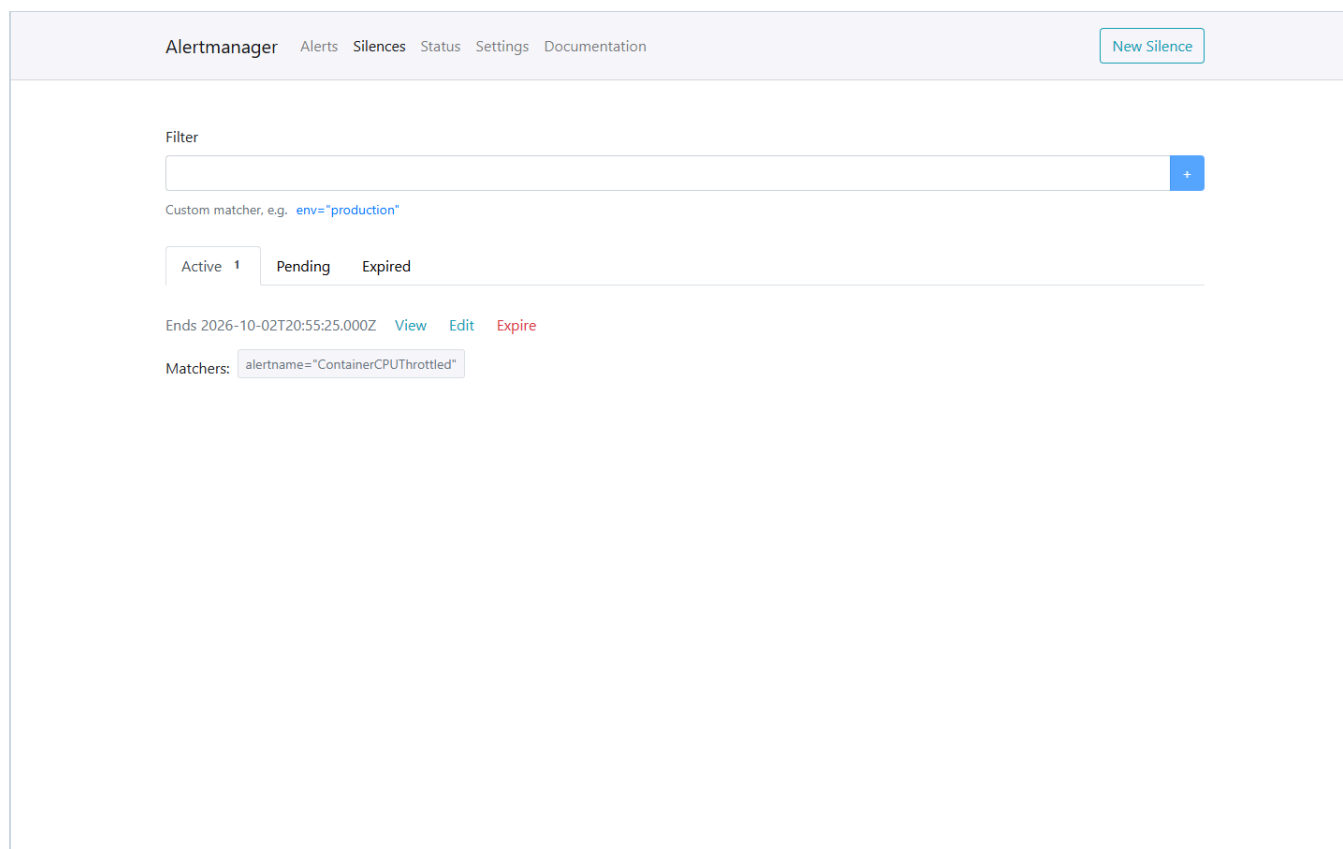
p95 latency over the test runs: the slow and hang scenarios stand out



Share of failed requests over the test runs



Black-box probe results for /health and /ready



A silence muting ContainerCPUThrottled during a planned load test

6. Comparison and evaluation

6.1 Detection and recovery times

`scripts/measure_detection.py` injected each failure in turn on a laptop (Docker Desktop, 20 CPUs, 8 GB RAM for Docker). For each one it recorded the time until the alert reached the ops chat and, for critical alerts, the on-call inbox, then the time from fixing the failure to the RESOLVED message.

Failure injected	Alert	Severity	Ops chat	On-call email	Resolved after fix
API error spike (50% of requests fail)	HighErrorRate	critical	64 s	59 s	57 s
Slow responses (1.5 s per request)	HighLatencyP95	warning	90 s	–	56 s
App hangs (3 s, health probe times out)	EndpointDown	critical	60 s	55 s	26 s
Memory leak up to ~85% of the limit	ContainerMemoryNearLimit	warning	72 s	–	26 s
CPU burn (container at its CPU limit)	ContainerCPULimited	warning	76 s	–	56 s
Database outage (docker stop ia2-redis)	DependencyDown	critical	32 s	27 s	26 s
Out of memory (fast leak, kernel OOM kill)	ContainerOOMKilled	critical	31 s	26 s	296 s ¹
Container crash (process exits)	ContainerRestarted	warning	16 s	–	– ²
Whole service down (docker stop ia2-shop-api)	ServiceDown	critical	52 s	47 s	27 s

¹ `ContainerOOMKilled` looks at OOM events over the last 5 minutes, so it stays visible for 5 minutes after the kill by design.

² A restart is a one-off event; the alert clears by itself 5 minutes later.

Every scenario was detected. The detection time is roughly the sum of four delays: the scrape interval (5 s; 10 s for cAdvisor), the time the query needs to cross its threshold (a 1-minute `rate()` window reacts gradually), the rule's `for` duration, and Alertmanager's `group_wait`.

- **Fastest:** a container crash (16 s). `ContainerRestarted` has no `for` duration, because a restart is a fact and not a trend.
- **Slowest:** slow responses (90 s). The p95 needs time to climb inside its 1-minute window, and the rule then waits another minute, on purpose, so a short slow patch doesn't page anyone.
- **Email vs chat:** critical alerts reached the on-call inbox about 5 seconds before the chat, because the critical route's `group_wait` is 5 s against 10 s for the default route.
- **Recovery:** most alerts were marked resolved 25 to 60 seconds after the fault was removed.

These numbers can be tuned. Halving every `for` duration would roughly halve detection time and also let shorter blips through. We kept values that suit a demo while still filtering one-off spikes.

What the first run taught us. The raw log of the first full run is kept in `docs/results/run1.log`. Two scenarios failed in it, and both failures were useful:

1. *CPU burn*. The first rule compared CPU usage with the container's limit using a 30-second `rate()`. Because cAdvisor refreshes every 5 s and is scraped every 10 s, the ratio jumped between 40% and 100% and never stayed above 90% for the full 30 s. We switched to the **throttling ratio**: the share of CPU periods in which the kernel made the container wait. Both counters come from the same sample, so the ratio is smooth. It held at 100% for the whole burn, and the alert fired in 76 s on the re-run (`docs/results/run2-cpu-and-database.log`).
2. *Database outage*. `DependencyDown` *flapped*: it fired, cleared, and fired again minutes later while Redis was still down. The Redis exporter was waiting up to 15 s to connect to the missing host, longer than Prometheus's 4 s scrape timeout. So instead of `redis_up = 0` there was no sample at all, and an empty result looks like "resolved". The fix had two parts. We gave the exporter a 1 s connection timeout, and we made the rule fire on `redis_up == 0 or up{job="redis"} == 0`, because missing evidence must not count as "healthy". Detection dropped to 32 s, and a new rule test covers the case.

6.2 Noise reduction

Three incidents show what Alertmanager adds on top of Prometheus:

Incident	Alerts firing in Prometheus	Notifications sent	Suppressed by inhibition
Redis stopped	DependencyDown, HighErrorRate, ServiceNotReady, HighLatencyP95 (pending)	DependencyDown only (1 email + 1 chat message)	HighErrorRate, ServiceNotReady, HighLatencyP95
OrderFlow stopped	ServiceDown, EndpointDown, ServiceNotReady, ContainerRestarted	ServiceDown (email + chat); ContainerRestarted (chat, after the restart)	EndpointDown, ServiceNotReady
App hangs	EndpointDown, ServiceNotReady	EndpointDown (email + chat)	ServiceNotReady

In each case the person on call gets one message naming the cause, instead of three or four messages about its symptoms.

The Prometheus Alerts page displays a list of alert rules grouped by category. The top navigation bar includes 'Query', 'Alerts', and 'Status'. Below the navigation bar, there are filters for 'Filter by rule group state' and 'Filter by rule name or labels'. The alert rules are organized into four groups: 'availability', 'containers', 'meta', and 'performance'. Each group shows the rule name and its current status (FIRING, INACTIVE, or PENDING).

Group	Rule Name	Status
availability	ServiceDown	INACTIVE (2)
	EndpointDown	INACTIVE (2)
	DependencyDown	FIRING (1)
	ServiceNotReady	FIRING (1)
	ContainerRestarted	FIRING (1)
containers	ContainerMemoryNearLimit	INACTIVE (3)
	ContainerOOMKilled	INACTIVE (3)
	ContainerCPUTHrottled	INACTIVE (3)
meta	Watchdog	FIRING (1)
performance	HighErrorRate	FIRING (1)
	HighLatencyP95	PENDING (1)

Prometheus during the Redis outage: three alerts firing, one pending

The Alertmanager interface shows a list of alert rules and their status. The top navigation bar includes 'Alertmanager', 'Alerts', 'Silences', 'Status', 'Settings', and 'Documentation'. A 'New Silence' button is located in the top right corner. Below the navigation bar, there are filters for 'Filter' and 'Group'. The alert rules are organized into a list of groups, each with a '+' icon to expand it. The groups are: 'ops-chat', 'heartbeat', 'oncall-email', 'ops-chat', 'oncall-email', 'ops-chat', 'ops-chat', and 'ops-chat'. Each group shows the alert name and its status (FIRING, INACTIVE, or PENDING).

Group	Alert Name	Status
ops-chat	alertname="ContainerRestarted"	1 alert
heartbeat	alertname="Watchdog"	1 alert
oncall-email	alertname="DependencyDown"	1 alert
ops-chat	alertname="DependencyDown"	1 alert
oncall-email	alertname="HighErrorRate"	1 alert
ops-chat	alertname="HighErrorRate"	1 alert
ops-chat	alertname="HighLatencyP95"	1 alert
ops-chat	alertname="ServiceNotReady"	1 alert

Alertmanager with "Inhibited" ticked: the symptoms are held back

6.3 False positives

We stopped OrderFlow for 10 seconds and started it again. **No critical page was sent** (0 `ServiceDown` or `EndpointDown` notifications). The `for: 30s` durations absorbed the blip. A warning-level `ContainerRestarted` message did appear, and correctly so: the container really had been restarted. The same behaviour is pinned down by a `promtool` unit test (“a 10 second blip does not alert anyone”), so a future change to the rule cannot silently undo it.

6.4 Resource use

Measured with `docker stats` while traffic was flowing and no faults were active:

Container	CPU	Memory
ia2-prometheus	2.2%	77 MiB
ia2-shop-api	2.8%	46 MiB
ia2-alertmanager	0.4%	34 MiB
ia2-cadvisor	3.4%	30 MiB
ia2-blackbox	1.0%	26 MiB
ia2-mailpit	0.0%	24 MiB
ia2-ops-chat	0.0%	14 MiB
ia2-traffic	1.2%	14 MiB
ia2-redis-exporter	0.8%	13 MiB
ia2-redis	0.7%	10 MiB

The monitoring side (Prometheus, Alertmanager, cAdvisor and the two exporters) used about **180 MiB of memory and under 10% of one CPU core** in total, while holding about 3,080 active time series. That is small next to typical application containers and fits easily on a laptop or a small VM.

6.5 Comparison with other tools

	Prometheus + Alertmanager	Nagios Core	Zabbix	Datadog	Grafana Alerting	AWS CloudWatch
Licence / cost	Open source (Apache 2.0)	Open source (GPL); Nagios XI is paid	Open source (AGPL since 7.0)	Commercial SaaS, priced per host and product	Open source (AGPL) or Grafana Cloud	Pay per metric, alarm and API call
How data is collected	Pull over HTTP; exporters	Active checks by plugins	Agents (push or pull), SNMP	Agent sends to SaaS	Queries other data sources	AWS services and agent push
Fit for containers	Native: Docker/Kubernetes discovery, cAdvisor	Weak: static host definitions	Docker template via Agent 2	Strong: agent auto-discovers containers	As good as its data source	Container Insights for ECS/EKS
Alert conditions	PromQL over any metric	Plugin thresholds (OK/WARN/CRIT)	Trigger expressions	Monitors with a query language	Data source query + conditions	Thresholds, anomaly detection, metric math
Routing and noise control	Grouping, inhibition, silences, routing tree	Contacts, escalations, host dependencies	Actions, escalations, dependencies	Notification rules, downtimes	Notification policies (Alertmanager-based)	Alarms to SNS topics
Configuration as code	YAML in Git, <code>promtool</code> / <code>amtool</code> tests	Text config files	Mostly web UI (API available)	UI, API, Terraform	UI and provisioning files	CloudFormation, Terraform
Logs and traces	No (pair with Loki, Tempo)	No	Limited	Yes	Through Loki, Tempo	Yes (CloudWatch Logs, X-Ray)
Long-term storage	Local TSDB; Thanos or Mimir for long-term	n/a	SQL database	Managed	Depends on source	Managed

Summarised from each product's public documentation (October 2026); pricing and licences change, so check the current terms.

For our goal (container failures, self-hosted, free, rules in Git), Prometheus with Alertmanager fits best. Datadog would be quicker to start and covers logs and traces, but it costs money per host and ties the project to one vendor. Grafana Alerting is a good front end but builds on the same kind of data source. Nagios and Zabbix come from a world of long-lived servers and fit short-lived containers less naturally.

6.6 Strengths and limitations we observed

Strengths

- One query language for graphs, recording rules and alerts. Ratios, percentiles and “relative to the container limit” are one line each.
- Alerting is code. It is reviewed in Git, unit-tested with `promtool`, and checked end to end in CI.

- Alertmanager turned an incident with several symptoms into one message about the cause, and the heartbeat makes a broken alerting path visible.
- A small footprint for what it does (section 6.4).

Limitations

- **Metrics only.** Finding *why* the error rate rose needs logs (Loki, ELK) or traces (Tempo, Jaeger).
- **Detection is deliberately not instant.** It is scrape interval + `for` duration + group wait. Faster settings mean more false alarms.
- **One Prometheus is a single point of failure.** High availability needs two Prometheus servers scraping the same targets and a clustered Alertmanager. Long-term storage needs Thanos or Mimir.
- **PromQL has a learning curve.** Our `or ... * 0` and `> 0` fixes are typical traps.
- **Cardinality.** cAdvisor exports many labels per container; without filtering, the number of series grows quickly.
- **Pull needs network reach.** Short batch jobs need the Pushgateway.

7. Testing and continuous integration

What	How
OrderFlow (20 tests)	pytest with an in-memory store instead of Redis; covers the API, readiness, metrics and every fault
Alert and recording rules	<code>promtool test rules</code> , 8 test cases including the false-positive, no-limit and CPU-throttling cases
Alertmanager routing	<code>amtool config routes test</code> for critical, warning and Watchdog alerts
Chat receiver (6 tests)	Python unittest
End to end	GitHub Actions starts the full stack, waits for all targets and the heartbeat, injects a 50% error rate, and passes only when the alert reaches both the chat and the email inbox

8. Conclusion

Prometheus and Alertmanager detected every failure we injected, from a clean outage to a hung process, a slow memory leak and a database failure hidden behind a running web service. Each was turned into a clear notification on the right channel within about a minute, followed by a resolved message. The most useful lesson was that detection alone is not enough. Grouping, inhibition and sensible `for` durations make the difference between one useful message and a flood. The tool's limits (metrics only, single-node storage, a query language to learn) are real but well known, and the ecosystem has standard answers for each of them.

9. References

1. Prometheus documentation: concepts, configuration, recording and alerting rules, unit testing.
<https://prometheus.io/docs/>
2. Alertmanager documentation: routing, grouping, inhibition, notification templates.
<https://prometheus.io/docs/alerting/latest/alertmanager/>
3. Blackbox exporter. https://github.com/prometheus/blackbox_exporter
4. cAdvisor (Container Advisor). <https://github.com/google/cadvisor>

5. Redis exporter. https://github.com/oliver006/redis_exporter
6. prometheus_client for Python. https://github.com/prometheus/client_python
7. Mailpit. <https://mailpit.axllent.org/>
8. B. Beyer et al., *Site Reliability Engineering*, O'Reilly, 2016: chapter 6, "Monitoring Distributed Systems".
9. Nagios, Zabbix, Datadog, Grafana and AWS CloudWatch product documentation (accessed October 2026).